AI Engineering and Claude Architect · Part I, Foundations

Generative AI and LLMs, Lesson 3

The Transformer End to End

A model with 124,439,808 parameters sounds unknowable. In fact, you can derive that exact number with arithmetic, part by part, and learn where the size goes. Most of it isn't in the attention.

Module	AI02 of 13
Lesson	3 of 7
Prerequisites	AI01 L3, AI02 L2
Exam relevance	Out of scope
Reading time	~40 minutes

Contents

1. The Transformer End to End

Lesson 3 of 7 · Generative AI and LLM Fundamentals

The Transformer End to End

Key idea

A transformer is one block repeated many times: attention lets tokens exchange information, and a feed-forward network lets each token process what it received. Residual connections and normalization keep that deep stack trainable. An LLM is this stack, a causal mask, and a final layer that scores every word in the vocabulary. **Published open models show that most parameters sit in the feed-forward layers and embeddings, rather than in attention itself.**

> *Before: attention, masks and positions (AI02 L2), residual connections and normalization (AI01 L3). Teaches: encoder and decoder, the transformer block, decoder-only LLMs, parameter counting, why scale changes everything. Used by: AI02 L4 (training), AI02 L5 (inference), AI11 L1–L2 (fine-tuning and quantization).*

The hook: how big is a model, really?

A vendor says a model has "1.5 billion parameters". Your manager asks, "How much memory does that need? Where does it all go?"

You can answer exactly, for any published transformer, since it's only multiplication. Real output from `code/ai02-llm/TransformerSize.java`, using GPT-2's published configuration:

```
GPT-2 small: d=768, heads=12, d_k=64, layers=12
  token embeddings         38,597,376 (31.0%)
  position embeddings         786,432 ( 0.6%)
  attention, all           28,348,416 (22.8%)
  feed-forward, all        56,669,184 (45.5%)
  layer norms                  38,400 ( 0.0%)
  total parameters        124,439,808
  memory at 32-bit 0.50 GB, at 16-bit 0.25 GB
GPT-2 XL: d=1600, heads=25, d_k=64, layers=48
  token embeddings         80,411,200 ( 5.2%)
  position embeddings       1,638,400 ( 0.1%)
  attention, all          491,827,200 (31.6%)
  feed-forward, all       983,424,000 (63.1%)
  layer norms                 310,400 ( 0.0%)
  total parameters      1,557,611,200
  memory at 32-bit 6.23 GB, at 16-bit 3.12 GB
```

Key idea

The formula gives exactly the widely reported counts for these two published models. It also shows something surprising: the famous attention mechanism holds under a third of the parameters here, and the feed-forward layers hold much more. Attention's power comes from how it routes information, not from its size.

Anthropic doesn't publish Claude's architecture or parameter count. So everything measurable here comes from models with public configurations, such as GPT-2.

Intuition: a meeting followed by desk work

Picture a team analyzing a contract, with one person per word. Work proceeds in rounds:

1. **Meeting (attention):** everyone shares, and each person listens to whoever's relevant.
2. **Desk work (feed-forward):** everyone goes away alone and thinks over what they heard.

Repeat this for many rounds. Early rounds resolve simple things, like who "it" refers to. Later rounds build on those results. After the final round, one person makes the prediction.

The original design: encoder and decoder

The 2017 paper targeted translation, with two stacks. Its base model used 6 layers in each stack, `d_model` of 512, 8 heads and a feed-forward size of 2,048.

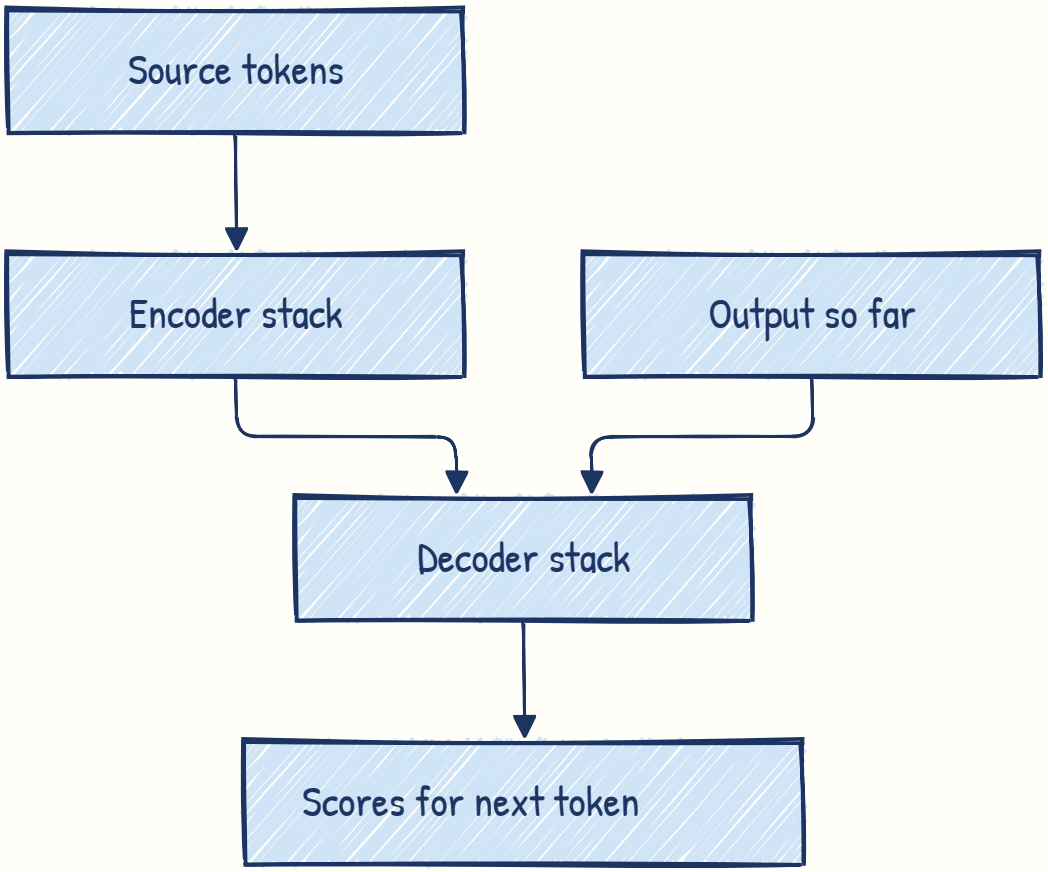


Figure 1. The original encoder-decoder. The encoder reads the source sentence, and the decoder generates the output while consulting the encoder through cross-attention.

Part	Attention types	Sees
Encoder	Self-attention, unmasked	The whole source sentence
Decoder	Masked self-attention, plus cross-attention	Output so far, plus the encoder's results

Three families

Family	Structure	Good for	Examples
Encoder-decoder	Both stacks	Translation, text-to-text	Original Transformer, T5
Encoder-only	No generation	Classification, embeddings	BERT
Decoder-only	Masked self-attention only	General text generation	GPT family

LLMs generally use a **decoder-only** design. Their input and output are one continuous sequence. Your prompt is just the beginning of a text the model continues, and that's why prompt and output share the same context window.

Inside one block

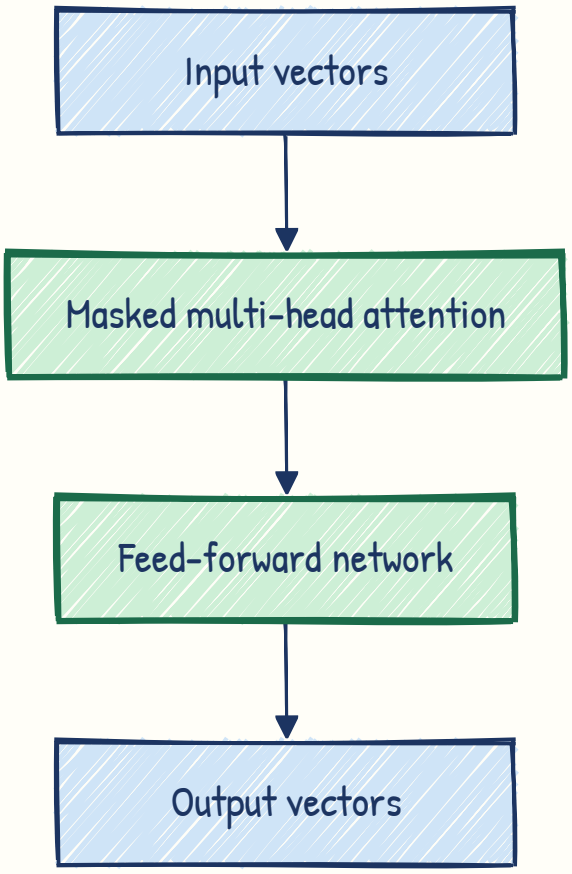


Figure 2. One decoder-only transformer block. Each of the two highlighted layers is also wrapped with layer normalization and a residual connection. A model stacks many of these.

Layer	Purpose	Why it's needed
Multi-head attention	Tokens exchange information	Provides context
Feed-forward network	Each token transforms its own vector, usually expanding to about 4× then back	Adds non-linearity and computing capacity. Attention is largely blending
Residual connection	Adds the input back in	Direct path for gradients (AI01 L3)
Layer norm	Stabilizes the range of values	Makes deep stacks trainable

Each block's output has the same shape as its input, `(tokens, d_model)`, which is why you can stack them freely.

The complete data flow

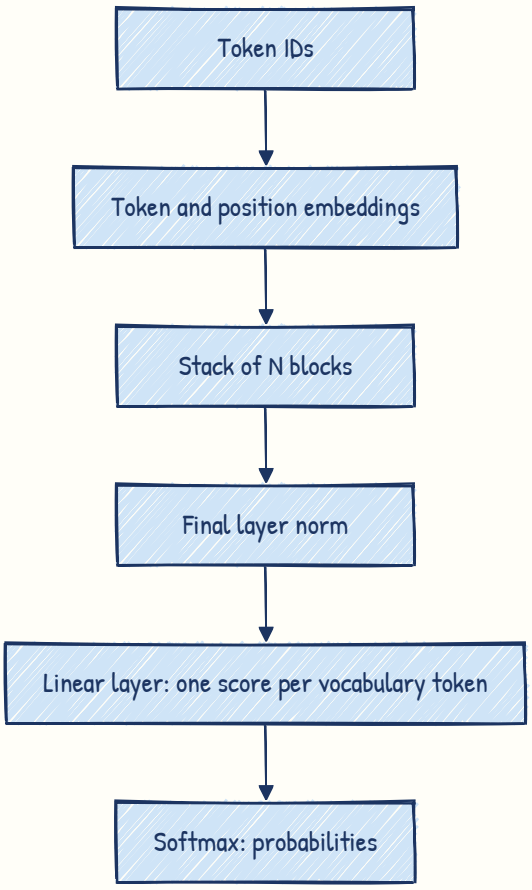


Figure 3. From token IDs to next-token probabilities. The shape stays the same through the stack, then the final layer scores the entire vocabulary.

Follow the shapes, for GPT-2 small with 10 input tokens:

Stage	Shape	Meaning
Token IDs	`(10)`	Ten integers
After embeddings	`(10, 768)`	One vector per token
After 12 blocks	`(10, 768)`	Same shape, far richer content
After linear layer	`(10, 50257)`	A score per vocabulary token, at each position
Used for generation	`(50257)`	Only the last position's scores

Weight tying is a neat detail: GPT-2's output layer reuses the embedding table's weights rather than having its own, saving those parameters. That's why the output layer adds nothing to the total above.

Deriving the parameter count

```
long tokenEmbedding = c.vocab() * d;
long positionEmbedding = c.context() * d;
// Query, key and value projections, then the output projection, with biases.
long attention = (d * 3 * d + 3 * d) + (d * d + d);
// Feed-forward layer: expands to 4d, then back to d, with biases.
long feedForward = (d * 4 * d + 4 * d) + (4 * d * d + d);
// Two layer norms per block, each with a scale and a shift per dimension.
long norms = 2 * (2 * d);
long block = attention + feedForward + norms;
long finalNorm = 2 * d;
```

Notice something in that code: **the number of heads doesn't appear in the parameter count**. Heads split the projections. They don't add to them. And `d` appears squared in the block calculations, so doubling the dimension roughly quadruples those layers.

Scale changes behavior

Comparison	GPT-2 small	GPT-2 XL	Change
Dimension	768	1,600	2.1×
Layers	12	48	4×
Parameters	124 M	1.56 B	12.5×

Why make models bigger? Because it keeps working, up to a point. Scaling Laws for Neural Language Models (Kaplan and colleagues, 2020, including several later Anthropic founders) reports that cross-entropy loss follows a power law in model size, dataset size and compute. Some trends hold across more than seven orders of magnitude.

> *Bigger isn't the same as better at what you want*
>
> InstructGPT reported that humans preferred outputs from a 1.3B-parameter aligned model over the 175B GPT-3. **Size buys capability. It doesn't buy following instructions** , and that needs alignment training (AI02 L4).

Why the obvious ideas fall short

Idea	Problem
Attention only, with no feed-forward	Attention mostly blends vectors, so non-linear processing would be limited
A very deep stack with no residuals or normalization	Hard to train (AI01 L3)
A separate vocabulary table per position	Enormous and wasteful. Sharing weights across positions is the key saving
Reading only the last few tokens	Loses long-range context, which is the whole point

How the field solved it

Problem	Solution	Source
RNNs are sequential	Attention-only blocks	Vaswani et al. 2017
Deep stacks are unstable	Residuals and layer norm	Standard, used in that paper
Training is slow	Adam with warm-up: 4,000 warm-up steps in the paper	Same paper
Models overconfident	Label smoothing and dropout, both 0.1 in the paper	Same paper
Which size to build?	Scaling laws	Kaplan et al. 2020

Anthropic's part: the transformer is shared field technology. Anthropic's distinctive work builds on it: alignment (AI02 L4), and interpretability (AI02 L6).

What this means in production

Forcing a new architecture here wouldn't help. Serving is covered in AI02 L5. These are the architecture facts you'll actually use.

Fact	What it means for you
Prompt and reply are one sequence	They share the context window. Reserve room for output (AI02 L1)
Output scores come from the last position	Generation proceeds one token at a time, which enables streaming
Weights are fixed after training	The model doesn't learn from your chats through inference. Memory must be built (AI03, AI06)
Parameters determine memory	At 16-bit, multiply parameters by 2 bytes to size self-hosting (AI11 L2)
Same weights, different context, different behavior	Prompts really do change how the computation flows

In the real world

You can check the arithmetic yourself: GPT-2's configuration is public, and the formula here reproduces its widely reported count of 124,439,808 parameters, and 1,557,611,200 for XL. HLD M21D extends this reasoning to fleet sizing: a 70B model at 16-bit needs 140 GB, which doesn't fit on one 80 GB GPU.

Java

Java is the wrong tool for training a transformer, and it's a fine one for reasoning about one. A capacity-planning helper could use this same formula.

```
record Config(
        String name, long vocab, long context, long dim, long layers, long heads) {
}
```

If you need	Typical path for a Java team
Capability	Call a hosted model (AI08)
Self-hosting	Run a dedicated Python or C++ server, and call it over HTTP or gRPC (AI11 L2)
Small models in the JVM	ONNX Runtime's Java API or DJL (AI01 L3)

Trade-offs

Decision	Gain	Cost
More layers	Deeper reasoning capacity	Latency and training difficulty
Larger dimension	Richer representations	Roughly quadratic parameter growth
Larger context	Longer inputs	Memory and compute per request (HLD M21D)
Decoder-only	One simple, general structure	Prompt and output compete for one window

When parameter counts don't matter: comparing hosted models. Evaluate them on your tasks, since a parameter count tells you little about fit (AI02 L7).

Common mistakes

1. **Assuming parameters equal quality.** InstructGPT shows otherwise.
2. **Thinking more heads add parameters.** They split the dimension.
3. **Believing the model learns from your conversation.** Weights don't change during inference.
4. **Assuming attention holds most of the weights.** The feed-forward layers hold more, in these published models.
5. **Claiming to know Claude's architecture.** Anthropic doesn't publish it. Say so in an interview.

What to say in an interview

For "walk me through a transformer", use the meeting-and-desk-work picture, then get specific: "Embeddings plus position, then N blocks of masked attention and feed-forward, each with residuals and normalization, then a linear layer over the vocabulary. In GPT-2 small, that's 124 million parameters, mostly feed-forward and embeddings." Giving a real number you can derive scores well.

Certification lens

How this lesson maps to CCAR-F

Not tested. Claude's internal architecture, training process and weights are explicitly out of scope. Skim this lesson if the exam is close, and return to it for your interviews and your production work.

These are original practice questions for understanding, not exam simulations.

Q Q1 · Calculation · Medium. A block has d = 512, using this lesson's formula. How many feed-forward parameters does one block have?

A $(512 \times 2048 + 2048) + (2048 \times 512 + 512) = 1{,}050{,}624 + 1{,}049{,}088 = \mathbf{2{,}099{,}712}$.

Q Q2 · Concept · Medium. Changing from 12 heads to 8 heads, with the same dimension, changes the parameter count by how much?

A It doesn't change at all, as long as the dimension is divisible by both. Heads split the same projections into different-sized pieces.

Q Q3 · Design decision · Hard. A team wants to size GPU memory for a model with 70 billion parameters at 16-bit. What do the weights need, and what's missing from that estimate?

A 70B × 2 bytes = **140 GB** for the weights alone. The estimate omits the KV cache, which grows with context length and batch size, and which often limits serving (AI02 L5, HLD M21D).

Revision - Lesson 3

→ A transformer block has attention (exchange information) and feed-forward (process it).

→ Each is wrapped by a residual connection and normalization, which keeps deep stacks trainable.

→ LLMs are generally decoder-only, so prompt and reply form one sequence in one window.

→ The final linear layer scores the whole vocabulary, using the last position to generate.

→ GPT-2 small has 124,439,808 parameters, and the formula reproduces this exactly.

→ Heads don't add parameters. Dimension has a roughly squared effect.

→ Most parameters there are in feed-forward layers and embeddings, not attention.

→ Loss follows power laws with scale (Kaplan 2020). But scale alone doesn't teach following instructions.

→ Weights are fixed at inference, so the model doesn't learn from your chats.

→ Anthropic doesn't publish Claude's architecture.

Your notes